CENTRO UNIVERSITÁRIO CATÓLICA DO TOCANTINS – UNICATÓLICA
CURSO DE ENGENHARIA DE SOFTWARE
HACKARUN

EQUIPE 21 – TYRANITAR
DIOGO DA SILVA FREITAS
GUSTAVO BARBOSA LIMA
GUSTAVO FERREIRA LIMA
KAUÃ AFRA PEREIRA BRITO

CAPI: ASSISTENTE DE INTELIGÊNCIA ARTIFICIAL DO CAPIM DOURADO SHOPPING

Planejamento técnico inicial da solução – Primeira entrega

PALMAS
2026

CAPI: ASSISTENTE DE INTELIGÊNCIA ARTIFICIAL DO CAPIM DOURADO SHOPPING

Planejamento técnico inicial da solução – Primeira entrega

Documento de planejamento técnico apresentado como primeira entrega do Hackarun, do curso de Engenharia de Software do Centro Universitário Católica do Tocantins – UniCatólica, referente ao Tema 04 – Integração da jornada de compra digital e física, proposto pelo Capim Dourado Shopping. Tecnologia sorteada para a equipe: design system.

PALMAS

2026

SUMÁRIO

1 INTRODUÇÃO.....3
2 PROPOSTA E ESTRUTURA INICIAL DAS TELAS.....3
2.1 Fluxo de navegação do aplicativo do cliente.....4
2.2 Aplicativo do cliente.....4
2.3 Painel da loja.....7
2.4 Painel do shopping.....8
2.5 Estados das telas.....8
3 DOCUMENTAÇÃO INICIAL DO PROJETO.....9
3.1 Requisitos funcionais.....9
3.2 Requisitos não funcionais.....10
3.3 Regras de negócio.....11
3.4 Modelagem UML.....12
3.5 Modelo de dados e API.....16
3.6 Plano de testes.....17
3.7 Organização da equipe.....18
4 LINGUAGEM DE PROGRAMAÇÃO ESCOLHIDA.....18
5 ARQUITETURA PROPOSTA.....19
5.1 Camadas do back-end.....19
5.2 Funcionamento da Capi.....19
5.3 Mapa 3D.....20
5.4 Segurança e LGPD.....20
5.5 Decisões arquiteturais e riscos.....21
6 PLATAFORMA E TECNOLOGIAS ADOTADAS.....21
7 INCORPORAÇÃO DA TECNOLOGIA SORTEADA: DESIGN SYSTEM.....22
7.1 Justificativa.....23
7.2 Identidade visual.....23
7.3 Conteúdo do sistema.....23
7.4 Incorporação ao código.....24
7.5 Regras de uso e validação.....24
8 CONSIDERAÇÕES FINAIS.....24
REFERÊNCIAS.....25

1 INTRODUÇÃO

Este documento apresenta o planejamento técnico inicial da solução desenvolvida pela Equipe Tyranitar para o Tema 04 – Integração da jornada de compra digital e física, proposto pelo Capim Dourado Shopping, localizado em Palmas (TO). O shopping concentra as lojas no pavimento térreo e a praça de alimentação no segundo pavimento.

O problema identificado é a fragmentação da jornada de compra: o consumidor pesquisa produtos, compara preços e lê avaliações na internet, mas conclui a compra presencialmente. Nessa transição, o shopping e os lojistas perdem a continuidade da jornada, os dados de interesse do cliente e a oportunidade de relacionamento.

Como solução, propõe-se a **Capi**, assistente de inteligência artificial do Capim Dourado Shopping, disponibilizada em um aplicativo web progressivo (*Progressive Web App* – PWA). O cliente conversa com a Capi por texto, foto ou *link*, encontra o produto nas lojas do shopping, faz a reserva, recebe a rota até a loja e, ao chegar ao balcão, o vendedor já sabe o que ele procura. Cada compra retorna ao aplicativo como pontos de fidelidade e como dados para lojistas e para a administração do shopping.

Para esta edição do Hackarun, cada equipe recebeu por sorteio uma tecnologia ou prática a ser incorporada à solução. A tecnologia sorteada para a Equipe Tyranitar foi o ***design system***, adotado como base visual e de código de todas as interfaces da Capi, conforme detalhado na seção 7.

A solução atende a três perfis de usuário, descritos no Quadro 1.

Quadro 1 – Perfis de usuário da solução

Perfil	**Quem é**	**Onde utiliza**
Cliente	Consumidor do shopping	Aplicativo web (PWA), no celular
Lojista	Gerente ou vendedor de uma loja	Painel da loja (/loja), no computador, celular ou *tablet* do balcão
Administração do shopping	Equipe de *marketing* e operação do shopping	Painel do shopping (/shopping)

Fonte: Elaborado pelos autores (2026).

As seções seguintes respondem, na ordem, aos itens exigidos para a primeira entrega: proposta e estrutura inicial das telas (seção 2), documentação inicial do projeto (seção 3), linguagem de programação escolhida (seção 4), arquitetura proposta (seção 5), plataforma e tecnologias adotadas (seção 6) e incorporação da tecnologia sorteada, o *design system* (seção 7).

2 PROPOSTA E ESTRUTURA INICIAL DAS TELAS

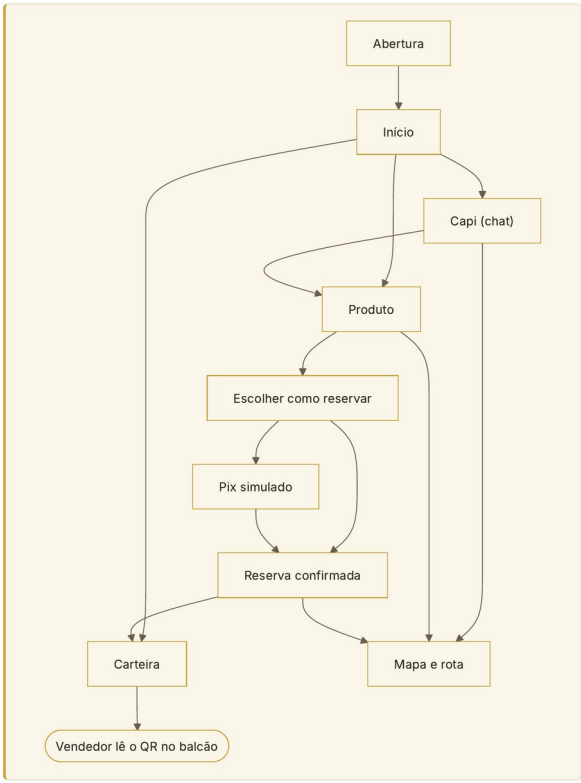
A solução é composta por três interfaces que compartilham a mesma identidade visual e os mesmos componentes: o aplicativo do cliente, com nove telas; o painel da loja, com cinco

telas; e o painel do shopping, com quatro telas. As telas já foram prototipadas com o *design system* em um protótipo navegável e animado, no qual é possível percorrer a jornada completa, da conversa com a Capi até a compra no balcão.

2.1 Fluxo de navegação do aplicativo do cliente

A Figura 1 apresenta o fluxo principal do cliente. Uma barra de navegação fixa no rodapé dá acesso às telas Início, Capi, Mapa e Carteira a partir de qualquer ponto do aplicativo.

Figura 1 – Fluxo de navegação do aplicativo do cliente



Fonte: Elaborado pelos autores (2026).

2.2 Aplicativo do cliente

O Quadro 2 descreve a finalidade de cada tela do aplicativo do cliente, os componentes do *design system* utilizados e os requisitos atendidos. As Figuras 2 a 4 mostram as telas no protótipo, com destaque para a conversa com a Capi e o fluxo de reserva.

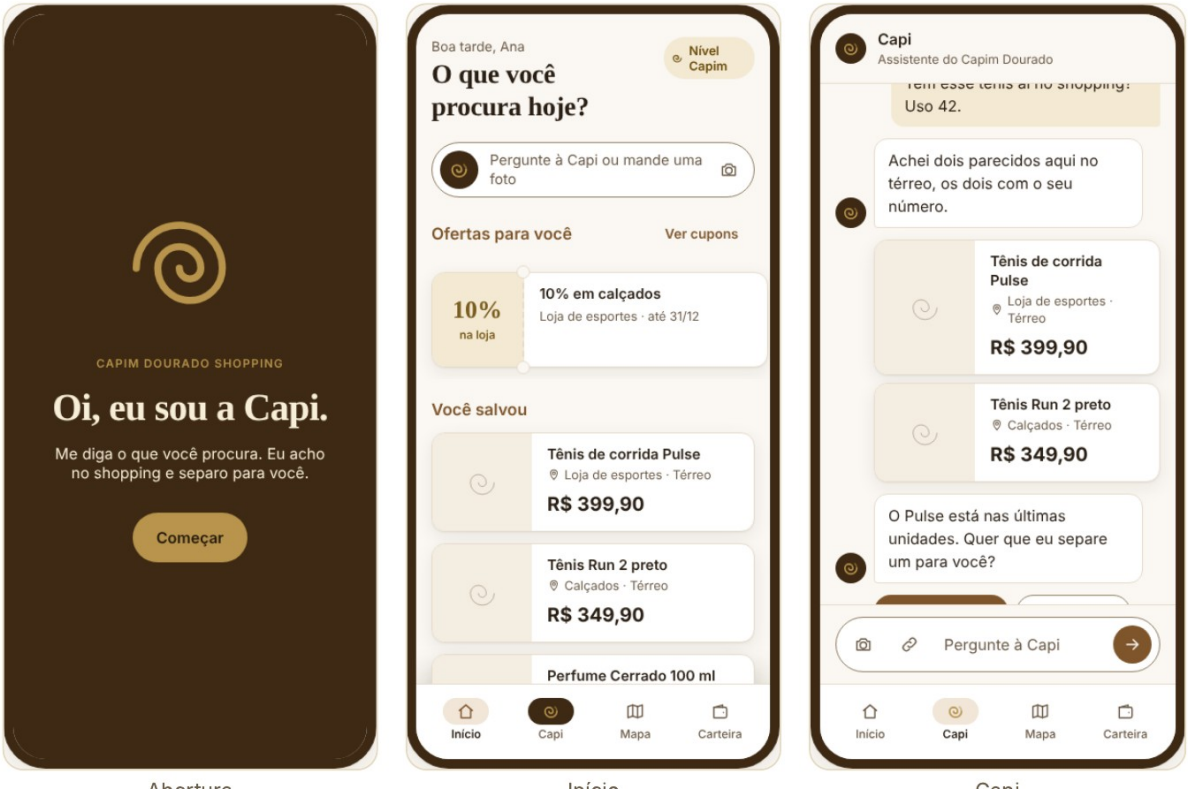
Quadro 2 – Telas do aplicativo do cliente

Tela	Finalidade	Componentes	Requisitos
Abertura	Apresentar a Capi, com a animação da trança de capim dourado	CapiMark, Button	RF01
Início	Ofertas para o cliente, produtos salvos e atalho para a Capi	NivelBadge, CupomTicket, ProdutoCard, BottomNav	RF06, RF16
Capi	Conversa por texto, foto ou *link*, com respostas em forma de produtos das lojas	ChatBubble, ChatComposer, ProdutoCard	RF02, RF03, RF04

Tela	Finalidade	Componentes	Requisitos
Produto	Loja, piso, preço, estoque, outras lojas com o item e botão de reservar	Badge, ProdutoCard, Button	RF05, RF07
Escolher como reservar	Painel inferior com as opções pagar agora por Pix ou pagar na loja	Button, Badge	RF08, RF09, RF13
Pix simulado	QR Code do Pix e confirmação do pagamento	Button	RF09
Reserva confirmada	Situação da reserva e próximos passos	ReservaStatus, Button	RF08, RF09
Mapa	Planta do térreo e da praça de alimentação, com a loja destacada e a rota desde a entrada	Badge	RF10
Carteira	QR Code do cliente, pontos, progresso de nível e cupons	CarteiraQR, ProgressoNivel, CupomTicket	RF11, RF12

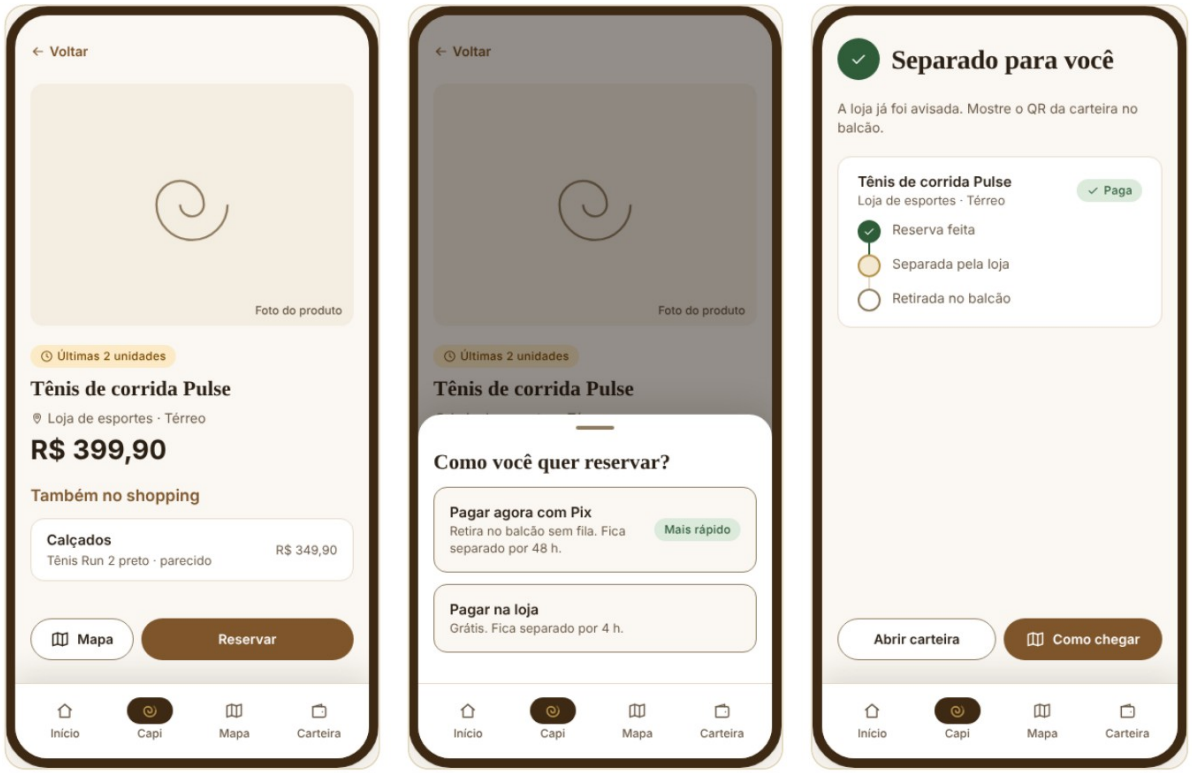
Fonte: Elaborado pelos autores (2026).

Figura 2 – Telas do aplicativo do cliente: abertura, início e conversa com a Capi



Fonte: Elaborado pelos autores (2026).

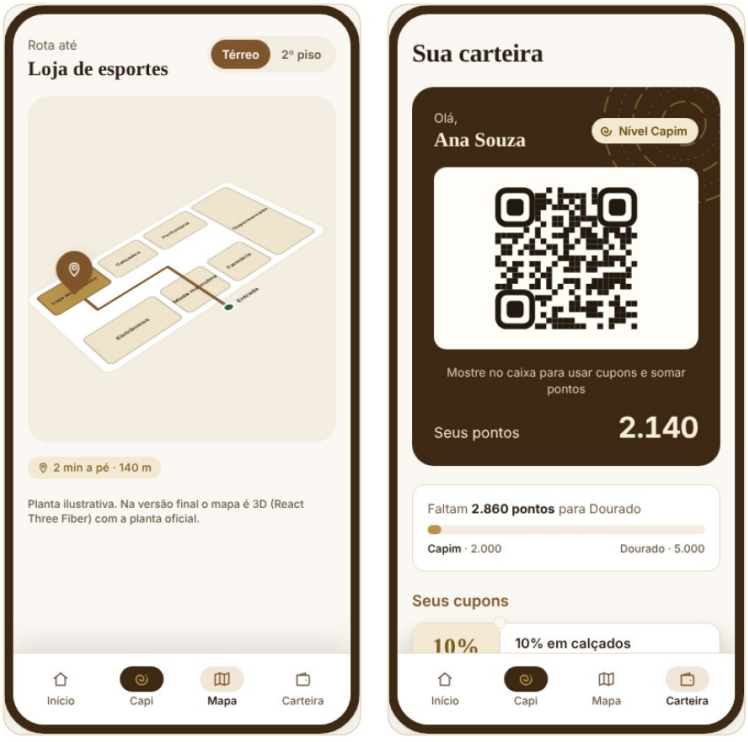
Figura 3 – Telas do aplicativo do cliente: produto, escolha da reserva e reserva confirmada



Produto | Escolher como reservar | Reserva confirmada

Fonte: Elaborado pelos autores (2026).

Figura 4 – Telas do aplicativo do cliente: mapa e carteira



Mapa | Carteira

Fonte: Elaborado pelos autores (2026).

O mapa do protótipo utiliza uma planta ilustrativa. Na versão final, a planta oficial do shopping (térreo e segundo pavimento) será convertida em SVG, cada loja receberá como identificador o seu número (por exemplo, loja-209) e o desenho será transformado em uma cena 3D.

2.3 Painel da loja

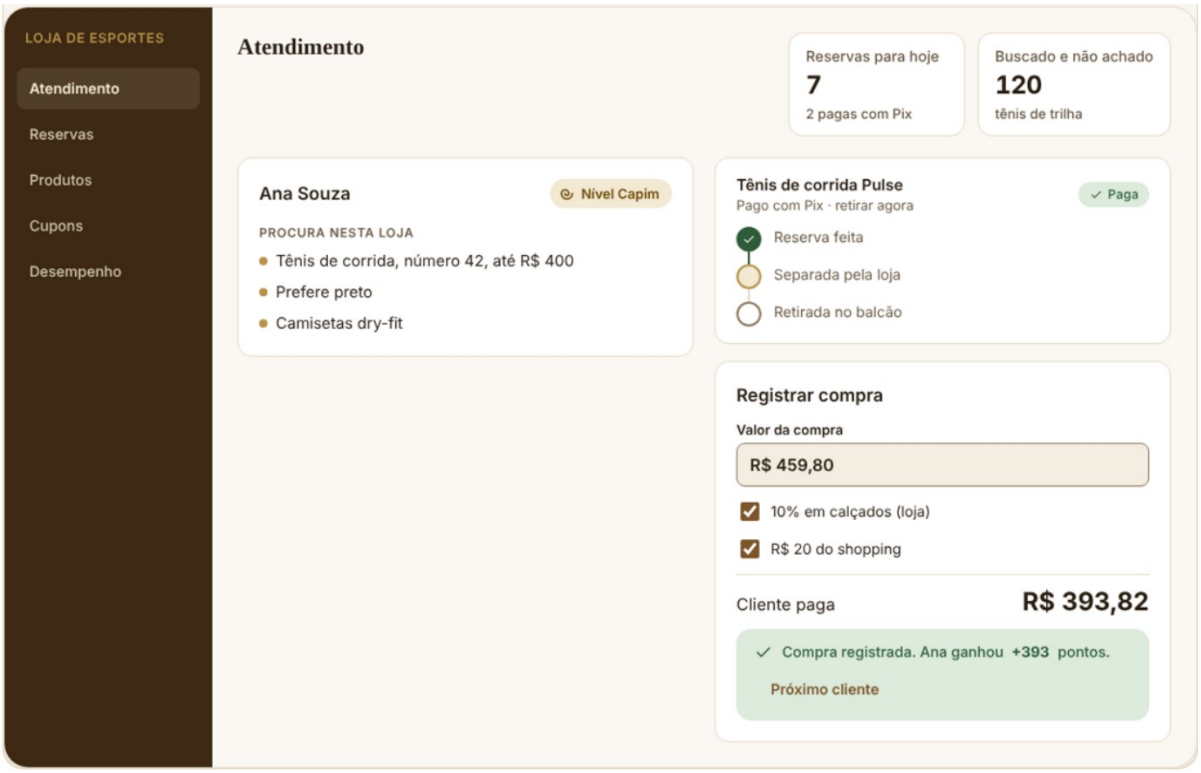
O painel da loja é utilizado pelo vendedor no balcão e pelo gerente na gestão do catálogo. Sua tela central é o Atendimento, mostrada na Figura 5: ao ler o QR Code da carteira do cliente, o vendedor vê a ficha de interesses (somente nas categorias que a loja vende e somente com o consentimento do cliente), as reservas e os cupons válidos, e registra a compra. O Quadro 3 resume as telas.

Quadro 3 – Telas do painel da loja

Tela	Finalidade	Componentes	Requisitos
Atendimento	Ler o QR Code do cliente, ver ficha e reservas e registrar a compra com cupons	FichaCliente, ReservaStatus, StatTile, Button	RF18, RF19
Reservas	Lista das reservas do dia por situação, com confirmar, separar e retirar	ReservaStatus, Badge	RF20
Produtos	Catálogo da loja, cadastro com atributos por categoria e importação de planilha	ProdutoCard, Button	RF21, RF22
Cupons	Criação de cupons da loja e acompanhamento do uso	CupomTicket, Button	RF23
Desempenho	Mais vendidos, mais buscados, buscados e não encontrados, horários de pico	StatTile	RF24, RF25

Fonte: Elaborado pelos autores (2026).

Figura 5 – Tela de atendimento do painel da loja



Fonte: Elaborado pelos autores (2026).

2.4 Painel do shopping

Quadro 4 – Telas do painel do shopping

Tela	Finalidade	Componentes	Requisitos
Lojas	Cadastro de lojas e lojistas	Button, Badge	RF27
Fidelidade e campanhas	Níveis, *vouchers* e campanhas da vitrine	NivelBadge, CupomTicket	RF28
Demanda	O que os clientes procuram e não encontram em nenhuma loja	StatTile	RF29
Jornada e reembolsos	Conversão das reservas e reembolsos devidos às lojas	StatTile	RF30, RF31

Fonte: Elaborado pelos autores (2026).

2.5 Estados das telas

Toda tela prevê quatro estados, todos construídos com os componentes do *design system*:

a) **carregando:** esqueleto no formato do conteúdo, sem tela em branco;
b) **vazio:** frase curta que orienta o próximo passo, por exemplo, “Você ainda não salvou nenhum produto. Pergunte à Capi.”;
c) **erro:** explicação do que ocorreu e botão para tentar novamente; se a Capi ficar indisponível, a busca comum continua funcionando (RNF10);

d) **com dados:** estado apresentado no protótipo.

3 DOCUMENTAÇÃO INICIAL DO PROJETO

A documentação inicial reúne os requisitos, as regras de negócio, a modelagem UML, o modelo de dados, o contrato da API, o plano de testes e o guia de trabalho da equipe. Cada requisito recebe um código (RF, RNF ou RN) que é referenciado na modelagem, na API e nos casos de teste, garantindo rastreabilidade.

3.1 Requisitos funcionais

Os requisitos foram priorizados em três níveis: **MVP**, que entra na demonstração do *hackathon*; **Desejável**, que entra se houver tempo; e **Futuro**, apresentado como evolução no *pitch*. O Quadro 5 lista os 31 requisitos funcionais.

Quadro 5 – Requisitos funcionais

Código	Requisito	Perfil	Prioridade
RF01	Cadastrar-se e entrar com e-mail e senha	Cliente	MVP
RF02	Conversar com a Capi por texto e receber produtos do catálogo como resposta	Cliente	MVP
RF03	Enviar foto ou captura de tela à Capi e receber produtos iguais ou parecidos das lojas	Cliente	MVP
RF04	Enviar o *link* de um produto à Capi e receber produtos iguais ou parecidos	Cliente	Desejável
RF05	Buscar produtos em todas as lojas, com filtro por categoria, loja e faixa de preço	Cliente	MVP
RF06	Ver a vitrine de ofertas, com cupons das lojas e campanhas do shopping	Cliente	MVP
RF07	Ver a página do produto, com lojas, preço, estoque e avaliações	Cliente	MVP
RF08	Reservar um produto para pagar e retirar na loja	Cliente	MVP
RF09	Reservar um produto pagando antes por Pix (simulado) e retirar no balcão	Cliente	MVP
RF10	Ver o mapa 3D do shopping com a loja destacada e a rota até ela	Cliente	MVP
RF11	Ver a carteira: QR Code do cliente, cupons, pontos e nível de fidelidade	Cliente	MVP
RF12	Registrar uma compra lendo o QR Code da nota fiscal (NFC-e)	Cliente	Desejável
RF13	Autorizar ou revogar o compartilhamento da ficha de preferências com as lojas	Cliente	MVP
RF14	Receber notificações no aplicativo (reserva pronta, cupom novo, cupom a vencer)	Cliente	Desejável
RF15	Avaliar um produto depois de uma compra registrada	Cliente	Desejável
RF16	Salvar produtos como favoritos e ver o histórico de reservas e compras	Cliente	Desejável

Código	Requisito	Perfil	Prioridade
RF17	Entrar no painel da loja com e-mail e senha	Lojista	MVP
RF18	Ler o QR Code do cliente e ver a ficha, as reservas e os cupons válidos dele	Lojista	MVP
RF19	Registrar uma compra no caixa, informando o valor e os cupons aplicados	Lojista	MVP
RF20	Gerir as reservas da loja: confirmar, separar, marcar retirada e cancelar	Lojista	MVP
RF21	Cadastrar e editar produtos, com atributos que variam conforme a categoria	Lojista	MVP
RF22	Importar produtos por planilha CSV	Lojista	Desejável
RF23	Criar cupons da loja e acompanhar o uso	Lojista	MVP
RF24	Ver relatórios de desempenho (mais vendidos, mais buscados, não encontrados, produtos parados, picos, cupons, tíquete médio)	Lojista	MVP
RF25	Ver o extrato: vendas pela plataforma, descontos em cupons e reembolsos a receber	Lojista	Desejável
RF26	Sincronizar o catálogo pela API da loja (VTEX, Shopify, ERP)	Lojista	Futuro
RF27	Cadastrar lojas e lojistas	Shopping	MVP
RF28	Configurar a fidelidade (níveis e *vouchers*) e criar campanhas e cupons	Shopping	MVP
RF29	Ver a demanda: mais buscados e buscados e não encontrados em nenhuma loja	Shopping	MVP
RF30	Ver a jornada (reservas convertidas em retirada, atendimentos com ficha convertidos em venda) e o gasto por cliente	Shopping	Desejável
RF31	Ver os reembolsos devidos a cada loja e marcá-los como pagos	Shopping	Desejável

Fonte: Elaborado pelos autores (2026).

3.2 Requisitos não funcionais

Quadro 6 – Requisitos não funcionais

Código	Requisito	Verificação
RNF01	Aplicativo do cliente em PWA responsivo, pensado primeiro para celular e instalável na tela inicial	Teste manual em Android e iPhone; Lighthouse
RNF02	Rotas da API sem IA respondem em menos de 500 ms; a Capi responde em até 8 s	Testes de API e demonstração
RNF03	Hospedagem sem custo na escala da demonstração (planos gratuitos da Cloudflare); a IA é cobrada por uso, com limite de gasto configurado	Painéis da Cloudflare e da Anthropic
RNF04	Senhas armazenadas apenas como *hash* (PBKDF2 com sal); sessão por JWT com expiração de 12 horas	Testes de unidade e de API
RNF05	Cada rota confere o perfil do usuário; o lojista acessa apenas dados da própria loja	Testes de autorização
RNF06	LGPD: a ficha só é gerada e exibida com consentimento, pode ser revogada a qualquer momento e nunca expõe a conversa	Testes de API e de unidade

Código	Requisito	Verificação
RNF07	A chave da API de IA fica apenas no servidor e nunca é enviada ao navegador	Revisão de código e teste de *build*
RNF08	Telas principais em conformidade com a WCAG 2.1 nível AA	axe-core
RNF09	Mapa 3D a pelo menos 30 quadros por segundo em celular intermediário	Teste manual
RNF10	Se a IA falhar, a Capi responde com mensagem amigável e a busca comum continua	Teste com falha simulada
RNF11	Interface em português do Brasil, valores em reais e datas no fuso de Palmas (UTC−3)	Revisão e testes E2E
RNF12	Código em TypeScript, com testes executados na integração contínua a cada *push*	GitHub Actions

Fonte: Elaborado pelos autores (2026).

3.3 Regras de negócio

Quadro 7 – Regras de negócio

Código	Regra
RN01	Reserva para pagar na loja expira 4 horas após a criação, se não for retirada.
RN02	Reserva paga por Pix fica separada por até 48 horas.
RN03	O estoque é descontado na criação da reserva e devolvido quando ela expira ou é cancelada.
RN04	A ficha do cliente só aparece para a loja com consentimento e apenas com interesses das categorias que a loja vende.
RN05	Um cupom da loja pode ser combinado com um *voucher* do shopping; dois cupons da mesma loja na mesma compra, não.
RN06	Cada cupom tem limite de usos por cliente e data de validade.
RN07	O cliente ganha 1 ponto a cada R$ 1,00 pago, já descontados os cupons.
RN08	O nível é calculado pelos pontos dos últimos 12 meses: Semente (0), Broto (500), Capim (2.000) e Dourado (5.000); os limites são configuráveis.
RN09	Uma nota fiscal só pode ser registrada uma vez (chave de acesso única) e apenas de CNPJ de loja cadastrada.
RN10	Avaliação só após compra registrada naquela loja, uma vez por produto.
RN11	Todo *voucher* do shopping usado em uma loja gera um reembolso pendente do shopping para essa loja.
RN12	Toda busca sem resultado é registrada para o relatório de demanda, sem identificar o cliente.
RN13	O comparativo com a categoria só é exibido com pelo menos 3 lojas na categoria, para que nenhuma loja veja números de outra.
RN14	A Capi só oferece produtos existentes no catálogo e nunca informa preço ou estoque que não tenha vindo do banco de dados.

Fonte: Elaborado pelos autores (2026).

Ficam fora do escopo do MVP: compra *online* com entrega em domicílio (a retirada é sempre na loja), pagamento real por Pix ou cartão (o Pix é simulado na demonstração),

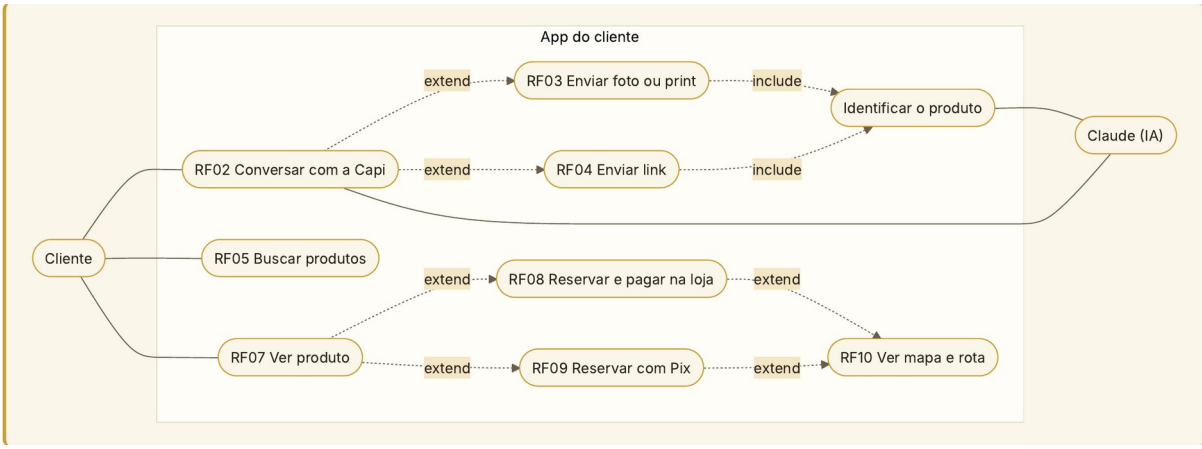
sincronização real com APIs das lojas (RF26) e notificação por proximidade ao chegar ao shopping, que exigiria aplicativo nativo.

3.4 Modelagem UML

A modelagem foi feita em Mermaid, o que permite versionar os diagramas junto ao código e visualizá-los diretamente no GitHub. Foram produzidos diagramas de casos de uso (divididos por perfil), diagramas de classes do domínio, diagramas de sequência dos fluxos críticos e o diagrama de estados da reserva. Os principais são apresentados a seguir.

A Figura 6 mostra os casos de uso centrais da jornada do cliente: a conversa com a Capi se estende ao envio de foto ou *link*, que incluem a identificação do produto pelo serviço de IA, e a visualização do produto se estende às duas formas de reserva e à rota no mapa.

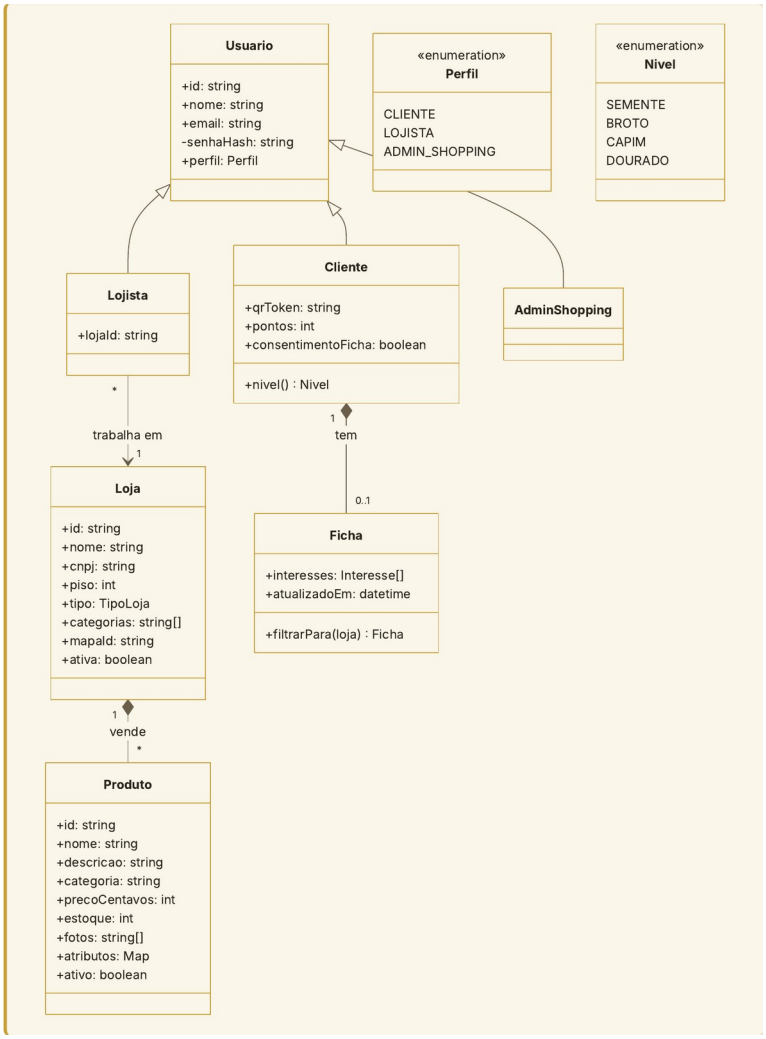
Figura 6 – Diagrama de casos de uso do cliente: descobrir e reservar



Fonte: Elaborado pelos autores (2026).

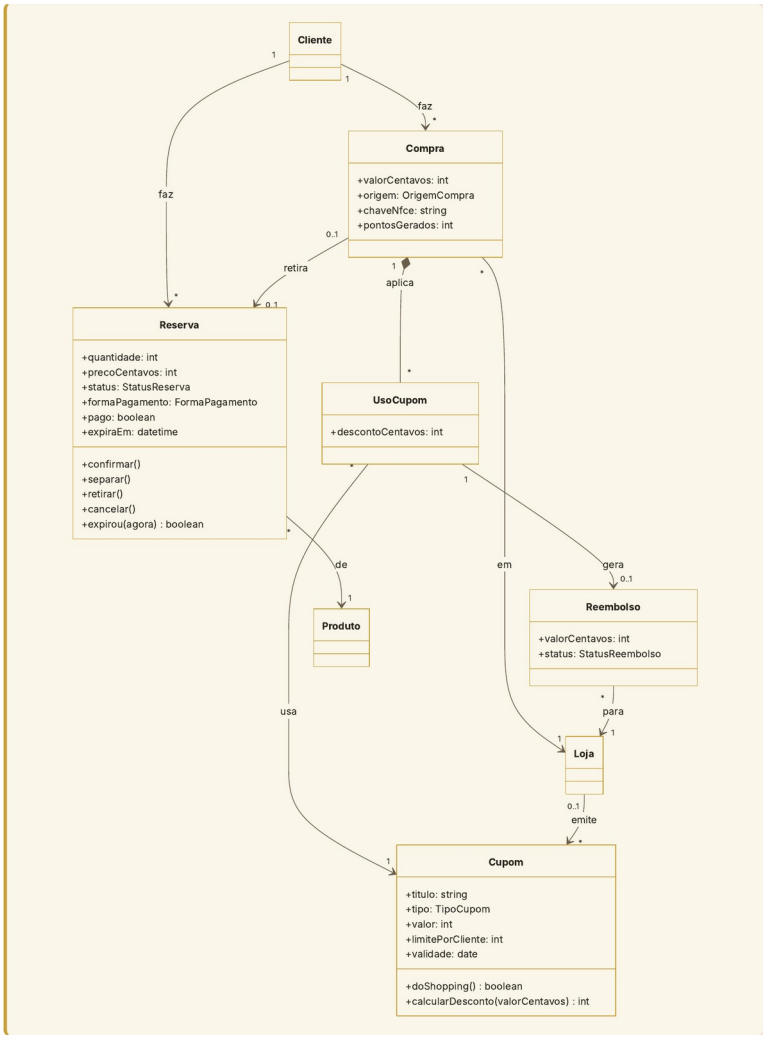
O domínio foi modelado em três grupos de classes. As Figuras 7 e 8 apresentam os dois primeiros: pessoas, lojas e catálogo, em que Cliente, Lojista e AdminShopping especializam Usuario; e reserva, compra e cupons, que concentram as regras da jornada de compra. O terceiro grupo (Mensagem, Busca e Avaliacao) registra a conversa com a Capi e alimenta os relatórios de demanda.

Figura 7 – Diagrama de classes: pessoas, lojas e catálogo



Fonte: Elaborado pelos autores (2026).

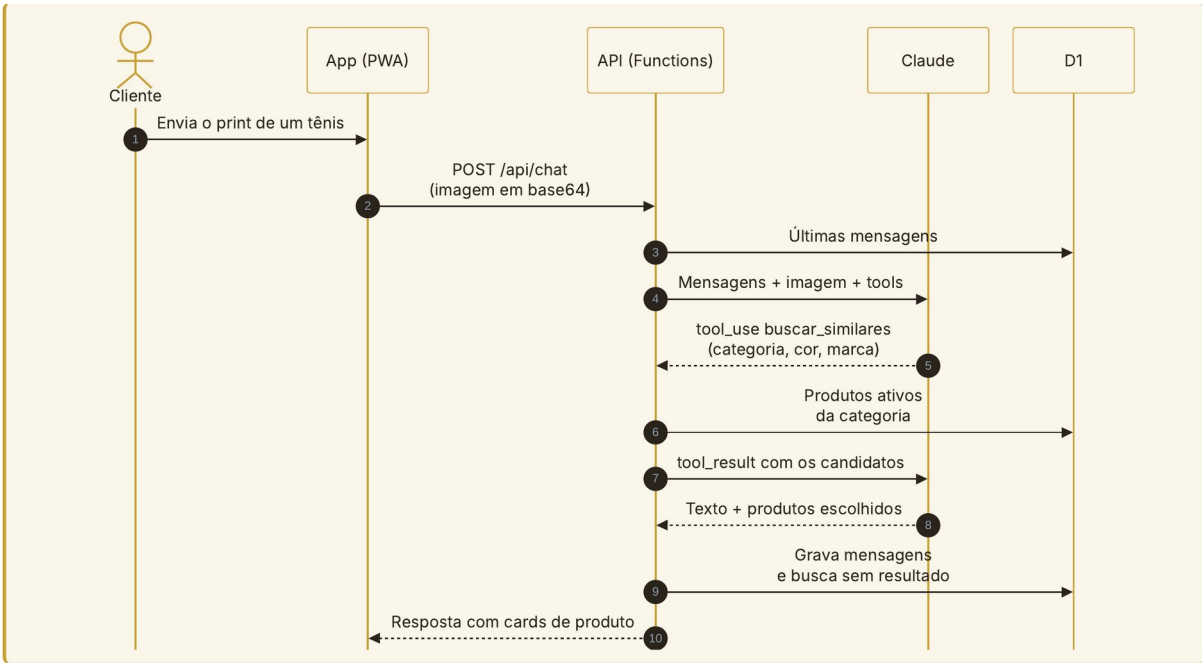
Figura 8 – Diagrama de classes: reserva, compra e cupons



Fonte: Elaborado pelos autores (2026).

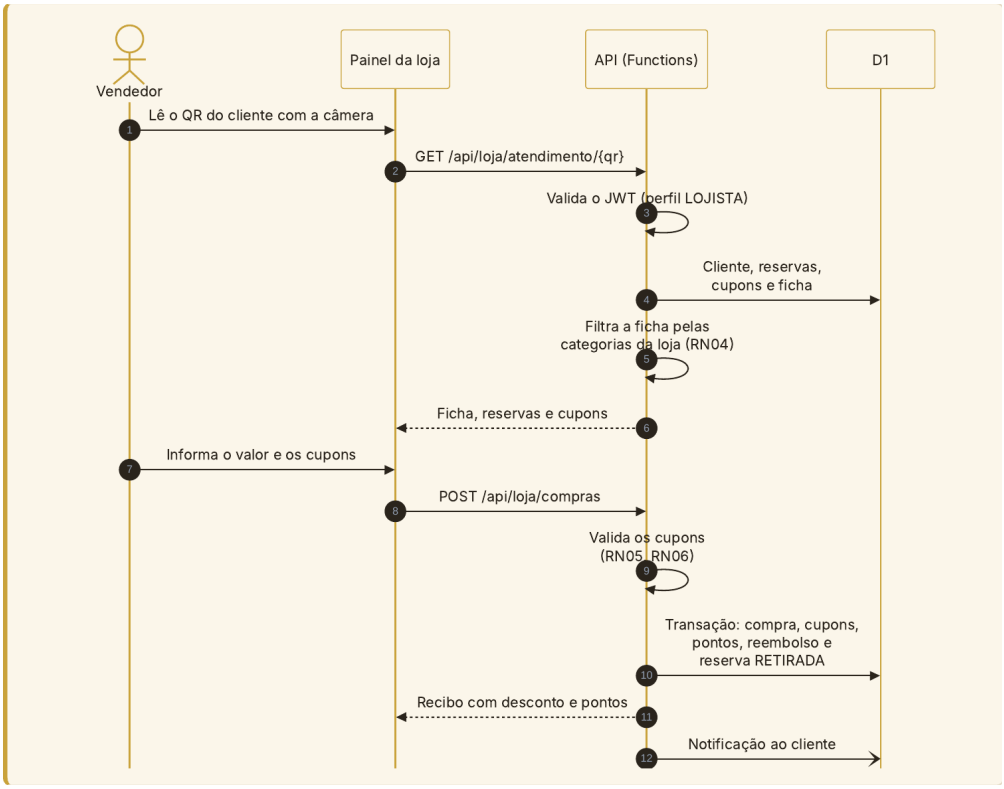
Os diagramas de sequência descrevem os fluxos que mais dependem da integração entre as partes do sistema. A Figura 9 mostra a busca por foto (RF03), em que a API envia a imagem ao modelo de IA, executa a ferramenta buscar_similares no banco e devolve os produtos escolhidos. A Figura 10 mostra o momento em que a jornada digital encontra a loja física: o vendedor lê o QR Code do cliente, recebe a ficha filtrada pelas categorias da loja e registra a compra com os cupons em uma única transação (RF18 e RF19).

Figura 9 – Diagrama de sequência da busca por foto (RF03)



Fonte: Elaborado pelos autores (2026).

Figura 10 – Diagrama de sequência do atendimento com QR Code e compra no caixa (RF18 e RF19)

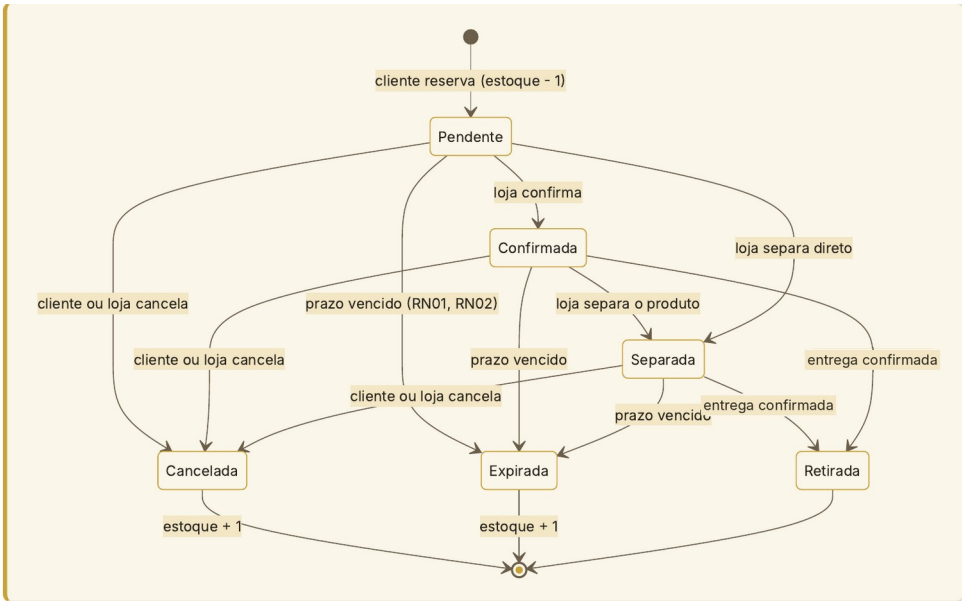


Fonte: Elaborado pelos autores (2026).

Também foram modelados os fluxos de reserva com Pix e rota (RF09 e RF10) e de registro por nota fiscal (RF12). Por fim, a Figura 11 apresenta o ciclo de vida da reserva: a criação desconta o estoque; a loja confirma e separa o produto; a leitura do QR Code no balcão

apenas abre o atendimento, e a retirada só é registrada quando o vendedor confirma a entrega do produto, de modo que consultar a ficha não altera a reserva; o cancelamento ou o vencimento do prazo devolvem o estoque. Como a plataforma de execução não dispõe de tarefas agendadas, a expiração é verificada sempre que a reserva é lida.

Figura 11 – Diagrama de estados da reserva



Fonte: Elaborado pelos autores (2026).

3.5 Modelo de dados e API

O banco de dados possui 15 tabelas, criadas por *migrations* versionadas. As restrições que o próprio banco consegue garantir (formatos, valores permitidos, unicidade e chaves estrangeiras) são declaradas como CHECK, UNIQUE e REFERENCES; as demais ficam nas regras de negócio da API. Valores monetários são armazenados como inteiros em centavos, datas em ISO 8601 (UTC) e identificadores como UUID. O Quadro 8 resume as tabelas.

Quadro 8 – Tabelas do banco de dados

Tabela	Finalidade
loja	Lojas do térreo e restaurantes da praça de alimentação, com CNPJ, piso, categorias e identificador no mapa
usuario	Clientes, lojistas e administradores, com *hash* da senha, perfil, QR Code, pontos e consentimento
produto	Catálogo de cada loja, com preço, estoque, fotos e atributos que variam por categoria (JSON)
reserva	Reservas com quantidade, preço no momento da reserva, situação, forma de pagamento e prazo
compra	Compras registradas no caixa ou por nota fiscal, com valor pago e pontos gerados
cupom, uso_cupom, reembolso	Cupons das lojas e *vouchers* do shopping, uso em cada compra e reembolsos devidos às lojas

Tabela	Finalidade
mensagem, ficha, busca	Conversa com a Capi, resumo de interesses do cliente e registro das buscas para os relatórios de demanda
avaliacao, favorito, notificacao	Avaliações comprovadas por compra, produtos salvos e notificações do aplicativo
configuracao	Parâmetros editáveis pelo shopping: limites dos níveis, prazos das reservas e pontos por real

Fonte: Elaborado pelos autores (2026).

Para impedir que dois clientes reservem a última unidade ao mesmo tempo, o desconto de estoque é feito por uma única instrução UPDATE condicionada à existência de estoque; se nenhuma linha for alterada, a API responde com conflito (409).

A API segue o padrão REST, utiliza JSON e fica sob o caminho /api no mesmo domínio do site. São cerca de 35 rotas, agrupadas em autenticação, catálogo, Capi, reservas, área do cliente (/api/me), painel da loja (/api/loja) e painel do shopping (/api/shopping). Todas as rotas não públicas exigem o cabeçalho Authorization com o *token* JWT, e os erros seguem um formato único, com código HTTP, identificador e mensagem em português (por exemplo, 409 SEM_ESTOQUE ou 422 CUPONS_INCOMPATIVEIS). O contrato completo é mantido no repositório, e o *front-end* utiliza dados simulados no mesmo formato enquanto as rotas são implementadas.

3.6 Plano de testes

A bateria de testes segue a pirâmide de testes: muitos testes rápidos de regras puras na base, menos testes de API e banco no meio e poucos testes de navegador no topo, concentrados nas jornadas da demonstração. Cada caso de teste é vinculado ao requisito que cobre. A Capi nunca chama o serviço de IA real nos testes automáticos: as respostas do modelo são gravadas e reproduzidas por um cliente simulado.

Quadro 9 – Níveis do plano de testes

Nível	O que garante	Ferramenta	Casos
Unidade (TU)	Regras puras: pontos, níveis, cupons, prazos, estados, ficha, NFC-e, senha e JWT	Vitest	30
Banco (TB)	A *migration* cria o banco e as restrições barram dados inválidos	Vitest + D1 local	15
API (TA)	Cada rota responde conforme o contrato e grava corretamente no banco	Vitest + vitest-pool-workers	36
Capi (TI)	O ciclo de ferramentas, os limites e as falhas da IA, com o modelo simulado	Vitest com respostas gravadas	8
E2E (TE)	Jornadas completas no navegador, em celular e computador	Playwright	10
Acessibilidade (TX)	Telas principais sem violações graves da WCAG 2.1 AA	axe-core	7

Nível	O que garante	Ferramenta	Casos
Manual (TM)	Desempenho do mapa, câmera, instalação do PWA e ensaio da demonstração	*Checklist*	6

Fonte: Elaborado pelos autores (2026).

3.7 Organização da equipe

Quadro 10 – Papéis da equipe

Papel	Responsabilidade
Desenvolvimento (*back-end* e Capi)	API, banco de dados, assistente de IA, implantação e integração
Desenvolvimento (*design* e *front-end*)	Telas do aplicativo e dos painéis, planta do shopping em SVG
Documentação e modelo de negócio	Documentação, dados reais das lojas e modelo de negócio
Pitch	Roteiro, gravação e edição do vídeo

Fonte: Elaborado pelos autores (2026).

O trabalho é dividido em quatro frentes (API e banco, Capi, aplicativo do cliente e painéis), cada uma com sua *branch*. Nenhum código é enviado diretamente à *branch* principal: tudo entra por *pull request*, com os testes aprovados na integração contínua. Os *commits* seguem o padrão tipo(escopo): descrição, em português, e toda alteração no banco é feita por uma nova *migration*.

4 LINGUAGEM DE PROGRAMAÇÃO ESCOLHIDA

A linguagem escolhida é o **TypeScript**, utilizado tanto no *front-end* quanto no *back-end*, com **SQL** (dialeto SQLite) para o banco de dados.

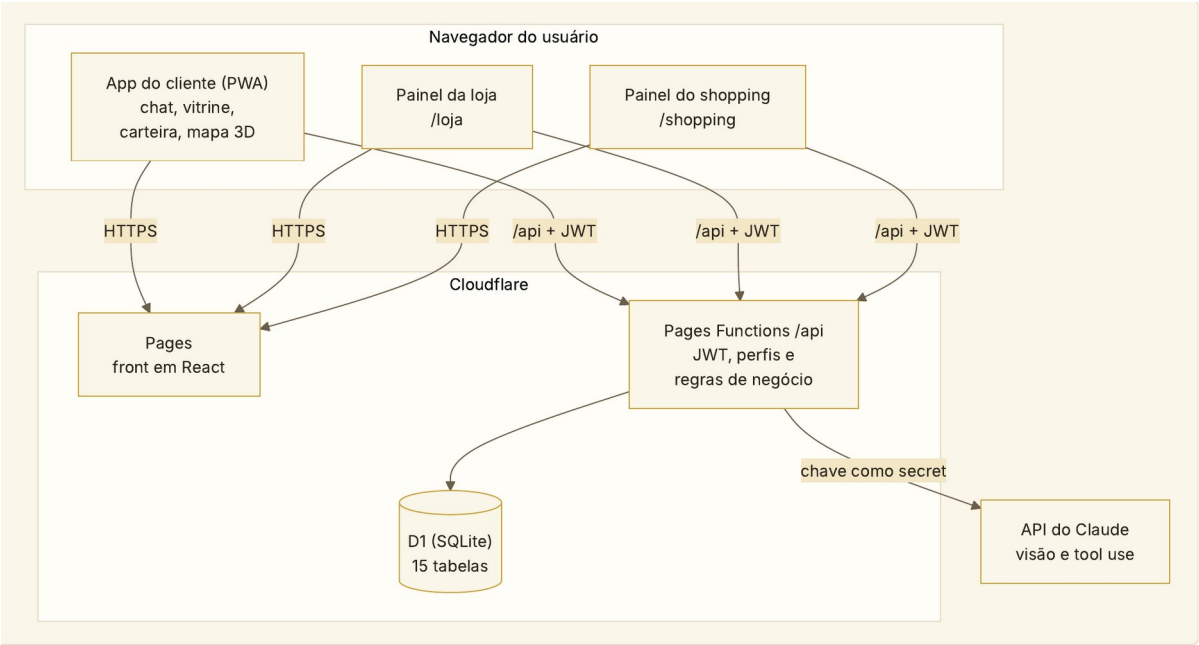
A escolha se justifica por quatro motivos:

a) **uma linguagem para todo o sistema:** a equipe trabalha no mesmo idioma no aplicativo, nos painéis e na API, o que facilita a troca de tarefas entre integrantes durante o *hackathon*;
b) **tipos compartilhados:** os tipos do contrato da API ficam em uma pasta comum (src/tipos) e são usados pelos dois lados, de modo que uma mudança em um campo é detectada pelo compilador antes de chegar à demonstração;
c) **compatibilidade com a plataforma:** o ambiente de execução escolhido (Cloudflare Pages Functions) executa TypeScript de forma nativa, sem servidor a configurar;
d) **ecossistema:** as bibliotecas adotadas (React, React Three Fiber, GSAP, Vitest e Playwright) possuem suporte completo a TypeScript.

5 ARQUITETURA PROPOSTA

A solução é uma aplicação web mantida em um único repositório e publicada em uma única implantação. O *front-end* (aplicativo do cliente e painéis) é servido pelo Cloudflare Pages; a API executa em Pages Functions no mesmo domínio; e os dados ficam no banco Cloudflare D1. A Capi é a única parte que sai da plataforma, chamando a API do Claude, da Anthropic, a partir do servidor. A Figura 12 apresenta a visão geral.

Figura 12 – Visão geral da arquitetura



Fonte: Elaborado pelos autores (2026).

5.1 Camadas do back-end

a) ***middleware*:** lê o cabeçalho Authorization, valida o JWT, coloca o usuário no contexto da requisição e responde 401 se o *token* estiver ausente ou vencido;
b) **rotas:** conferem o perfil (403 se não permitido), validam a entrada e chamam a regra de negócio;
c) **regras de negócio:** funções puras, sem acesso ao banco, testadas em unidade (cálculo de pontos e nível, combinação de cupons, expiração de reserva, filtro da ficha e leitura da chave da NFC-e);
d) **repositórios:** consultas SQL parametrizadas, nunca com texto concatenado; operações que alteram várias tabelas são executadas em uma única transação.

5.2 Funcionamento da Capi

A Capi funciona como um ciclo de uso de ferramentas (*tool use*): o modelo de IA recebe a conversa e a lista de ferramentas disponíveis, decide qual chamar, o servidor executa a ferramenta no banco de dados e devolve o resultado, repetindo o processo até que o modelo

responda em texto. A resposta chega ao cliente acompanhada de cartões de produto e de ações (reservar ou ver a rota). As ferramentas são:

a) **buscar_produtos:** busca por texto, categoria, faixa de preço e loja;
b) **buscar_similares:** recebe atributos (categoria, cor, marca, estilo) e devolve candidatos da mesma categoria, cabendo ao modelo escolher os mais parecidos;
c) **reservar:** cria a reserva para o cliente autenticado, somente após a confirmação dele na conversa;
d) **rota_na_loja:** devolve a loja, o piso e o identificador no mapa, para que o aplicativo desenhe a rota;
e) **gerar_ficha:** atualiza a ficha de interesses do cliente, somente se houver consentimento.

Quando o cliente envia uma foto, a imagem segue junto da mensagem e o modelo descreve o item antes de chamar buscar_similares. Quando envia um *link*, o servidor lê o título, a imagem e o preço da página e segue o mesmo fluxo. O prompt de sistema determina que a Capi só ofereça produtos vindos das ferramentas (RN14), peça confirmação antes de reservar e responda em português, de forma curta. Com um catálogo de cerca de 100 produtos na demonstração, filtrar por categoria e deixar o modelo escolher é suficiente; para catálogos maiores, a evolução prevista é a busca vetorial com Cloudflare Vectorize.

5.3 Mapa 3D

A planta de cada piso é desenhada em SVG, com um elemento por loja identificado pelo mesmo código registrado no banco. O carregador de SVG do Three.js lê a planta e a extrusão de geometria transforma cada loja em um bloco 3D clicável, com os dois pisos visíveis separados ou empilhados. A rota é calculada pelo algoritmo de Dijkstra sobre um grafo de pontos dos corredores e desenhada como uma linha animada da entrada até a loja. Para garantir desempenho em celulares, a cena usa poucos polígonos, cores sólidas e nenhuma sombra dinâmica.

5.4 Segurança e LGPD

a) senhas protegidas com PBKDF2 (100 mil iterações e sal aleatório por usuário);
b) JWT assinado com segredo armazenado como *secret* da plataforma, com expiração de 12 horas e o perfil no conteúdo do *token*;
c) toda consulta do painel da loja é filtrada pelo identificador da loja presente no *token*, nunca por um valor vindo da requisição;
d) a ficha só é gerada e lida com consentimento; revogar o consentimento apaga a ficha imediatamente;
e) o vendedor recebe apenas o resumo filtrado pelas categorias da loja, nunca a conversa;

f) os relatórios de demanda são agregados e não identificam quem buscou;
g) CORS restrito ao domínio do próprio sistema.

5.5 Decisões arquiteturais e riscos

Quadro 11 – Decisões arquiteturais

Decisão	Alternativa considerada	Motivo
Cloudflare Pages + Functions + D1	*Back-end* Java ou Node em servidor gratuito	Sem servidor hibernando na hora da demonstração, hospedagem gratuita e uma única implantação
React web (PWA)	React Native	A banca testa pelo *link*, o mapa 3D é mais simples no navegador e há um único código para aplicativo e painéis
Retirada sempre na loja	Compra *online* com entrega	O tema é integrar o digital ao físico, e o shopping ganha com o cliente presente
Similares escolhidos pelo modelo	*Embeddings* em banco vetorial	Catálogo pequeno na demonstração; busca vetorial fica como evolução
Pix simulado	Integração com provedor de pagamento	Fora do escopo do *hackathon*

Fonte: Elaborado pelos autores (2026).

Quadro 12 – Riscos e mitigações

Risco	Mitigação
Internet instável no evento	Respostas da Capi salvas para as imagens do *pitch* (RNF10)
Mapa 3D pesado no celular	Testar antecipadamente em celular intermediário e começar por um piso simples
Notificações no iPhone	*Web push* só funciona com o PWA instalado; na demonstração, usar as notificações internas do aplicativo
Migration não aplicada em produção	Aplicar as *migrations* antes de integrar o código que depende delas

Fonte: Elaborado pelos autores (2026).

6 PLATAFORMA E TECNOLOGIAS ADOTADAS

A plataforma de entrega é um **aplicativo web progressivo (PWA)**, utilizável no celular e no computador e instalável na tela inicial sem passar por lojas de aplicativos, o que permite à banca avaliadora testar a solução apenas acessando um *link*. A infraestrutura é a **Cloudflare**, cujos planos gratuitos atendem à escala da demonstração sem custo de hospedagem. O único item com custo variável é a API do Claude, cobrada por volume de *tokens* processados: o gasto é proporcional ao uso, baixo na escala da demonstração e controlado por um limite configurado na conta. O Quadro 13 detalha as tecnologias por camada.

Quadro 13 – Tecnologias adotadas por camada

Camada	Tecnologia	Motivo
Front-end	React, Vite, TypeScript e Tailwind CSS	Desenvolvimento rápido e tipado, com um único projeto para aplicativo e painéis
PWA	vite-plugin-pwa	Aplicativo instalável, sem loja de aplicativos
Mapa 3D	React Three Fiber e drei (Three.js)	3D declarativo em React, com controles de câmera prontos
Animações	GSAP com @gsap/react	Padrões de movimento da marca, com respeito à preferência por menos movimento
QR Code	qrcode.react (geração) e html5-qrcode (leitura pela câmera)	Funcionam no navegador, sem aplicativo nativo
Gráficos	Recharts	Gráficos dos painéis com pouco código
API	Cloudflare Pages Functions (TypeScript)	Mesma implantação do *front-end*, sem servidor para manter
Banco de dados	Cloudflare D1 (SQLite)	Gratuito na escala da demonstração, com *migrations* versionadas
Autenticação	JWT (HS256) com a biblioteca jose; senhas com PBKDF2 (Web Crypto)	Executa no ambiente da Cloudflare, sem dependência nativa
Inteligência artificial	API do Claude (Anthropic), com visão e *tool use*	Entende texto e imagem e aciona funções do sistema
Testes	Vitest, vitest-pool-workers, Playwright e axe-core	Pirâmide de testes completa
Integração contínua	GitHub Actions	Testes executados a cada *push* e *pull request*
Fontes	Fraunces e Figtree (Google Fonts)	Títulos com personalidade e interface legível

Fonte: Elaborado pelos autores (2026).

A implantação ocorre em três ambientes: **local**, na máquina de cada integrante, com banco D1 local; ***preview***, em que cada *branch* publicada recebe uma URL própria para revisão; e **produção**, atualizada a cada envio para a *branch* principal, após a aplicação das *migrations*. As chaves sensíveis (chave da API de IA e segredo do JWT) são cadastradas como *secrets* na plataforma e nunca são versionadas.

7 INCORPORAÇÃO DA TECNOLOGIA SORTEADA: DESIGN SYSTEM

A tecnologia sorteada para a equipe foi o ***design system***. Ele é a base visual de todas as telas: o aplicativo do cliente, o painel da loja e o painel do shopping utilizam os mesmos *tokens* e os mesmos componentes. O sistema completo, com prévias de cada componente, já está publicado e serviu de base para o protótipo apresentado na seção 2.

7.1 Justificativa

a) **três interfaces, uma identidade:** cliente, lojista e administração veem a mesma marca, e o botão de reservar é igual em qualquer tela;
b) **desenvolvimento em paralelo:** com os componentes prontos, quem monta uma tela apenas combina peças já testadas, sem redefinir cor, espaçamento ou tamanho;
c) **acessibilidade centralizada:** contraste, foco de teclado e textos para leitores de tela são resolvidos no componente e valem para todas as telas;
d) **manutenção em um único lugar:** se o shopping solicitar outro tom de cor, basta alterar um *token* e todas as telas acompanham.

7.2 Identidade visual

A paleta parte das cores do próprio Capim Dourado Shopping e da paisagem do Tocantins, conforme o Quadro 14. O padrão gráfico é a espiral trançada do artesanato de capim dourado: ela é o avatar da Capi, ganha uma volta a cada nível de fidelidade (Semente, Broto, Capim e Dourado) e aparece como textura na carteira. A tipografia combina Fraunces, serifada, para títulos, e Figtree, sem serifa, para a interface.

Quadro 14 – Paleta de cores do design system

Papel	***Token***	**Tema claro**	**Tema escuro**	**Origem**
Ação	marrom	#7F562C	#C99A66	Barra de menu e botões do site do shopping
Marca e conquista	ouro	#B8944C	#D1B06A	Dourado do logotipo do shopping
Destaque escuro	terra	#3D2914	#2C2116	Tom de terra do cerrado
Confirmação	cerrado	#2F5D3A	#86C095	Verde das veredas do Jalapão
Fundo	surface	#FBF8F3	#15110C	Creme do capim seco

Fonte: Elaborado pelos autores (2026).

7.3 Conteúdo do sistema

Quadro 15 – Elementos do design system

Parte	**Conteúdo**
Cores	38 *tokens* em tema claro e escuro, cada um com regra de uso; todos os pares de texto atingem contraste mínimo de 4,5:1
Tipografia	10 estilos: display, title-lg, title-md, title-sm, body, body-sm, label, caption, price e metric
Espaço e forma	Grade de 4 px (space-1 a space-12), 4 raios de borda e 2 sombras
Movimento	5 durações e 4 curvas de animação, com equivalentes no GSAP
Componentes	Button, Badge, NivelBadge, ProgressoNivel, ProdutoCard, ChatBubble, ChatComposer, CupomTicket, CarteiraQR, ReservaStatus, FichaCliente, StatTile e BottomNav
Guia da marca	Princípios, voz e conteúdo, uso de cada cor, iconografia e acessibilidade

Fonte: Elaborado pelos autores (2026).

7.4 Incorporação ao código

O *design system* não é apenas uma referência visual: ele é a fonte do código das telas, por meio do seguinte encadeamento:

a) **fonte única:** o arquivo tokens.json é o mesmo do *design system* publicado, e nenhum valor de cor, espaçamento ou fonte é escrito diretamente no código;
b) **variáveis CSS:** um *script* de *build* transforma os *tokens* em variáveis CSS, com o tema claro como padrão e o tema escuro aplicado conforme a preferência do sistema operacional;
c) **Tailwind CSS:** a configuração do Tailwind aponta para essas variáveis, de modo que as telas utilizam classes como bg-marrom e text-on-marrom;
d) **componentes React:** os 13 componentes são implementados em src/componentes, com os mesmos nomes e propriedades do *design system*, e as telas apenas os importam;
e) **movimento:** as animações usam GSAP e respeitam a preferência por movimento reduzido; os quatro movimentos da marca são a trança se desenhando, os cartões em cascata, os pontos somando e a mudança de nível.

7.5 Regras de uso e validação

Para manter a consistência, a equipe adota as seguintes regras: toda tela nova utiliza os componentes existentes, e um componente novo entra primeiro no *design system*, com prévia e regra de uso; o marrom é reservado para ação, o ouro para marca e conquista e o verde cerrado para confirmação, sem que um estado seja comunicado apenas pela cor; o botão principal aparece no máximo uma vez por tela; e qualquer mudança de *token* passa por *pull request* e atualiza o sistema publicado.

A conformidade é verificada por testes automatizados: um teste de unidade confere o contraste de todos os pares de texto nos dois temas; o axe-core verifica a acessibilidade das sete telas principais com os componentes montados; capturas de tela com Playwright detectam mudanças visuais não intencionais entre versões; e um teste E2E confirma que as telas funcionam com a preferência por movimento reduzido ativada.

8 CONSIDERAÇÕES FINAIS

Este planejamento define a base técnica da Capi: telas prototipadas para os três perfis de usuário, requisitos e regras de negócio rastreáveis até os testes, linguagem única em TypeScript, uma arquitetura enxuta, com hospedagem gratuita na Cloudflare e custo de IA proporcional ao uso e um *design system* que garante identidade, acessibilidade e produtividade à equipe.

As próximas etapas consistem na implementação dos requisitos priorizados como MVP, na conversão da planta oficial do shopping para o mapa 3D, na carga dos dados reais das lojas e na preparação do *pitch* para a entrega final.

REFERÊNCIAS

ANTHROPIC. **Claude documentation**: tool use. San Francisco: Anthropic, 2026. Disponível em: https://docs.claude.com. Acesso em: 13 nov. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**, Brasília, DF, 15 ago. 2018.

CLOUDFLARE. **Cloudflare D1 documentation**. San Francisco: Cloudflare, 2026. Disponível em: https://developers.cloudflare.com/d1/. Acesso em: 13 nov. 2026.

CLOUDFLARE. **Pages Functions documentation**. San Francisco: Cloudflare, 2026. Disponível em: https://developers.cloudflare.com/pages/functions/. Acesso em: 13 nov. 2026.

META OPEN SOURCE. **React documentation**. 2026. Disponível em: https://react.dev. Acesso em: 13 nov. 2026.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 13 nov. 2026.